

Learning Electrostatic Potential with U-Net: From Point Charges to Neural Field Solvers

LAUMeltFlow Project

December 15, 2025

Abstract

This report documents the development of a U-Net neural network that learns to compute electrostatic potential from point charge distributions in 2D. We begin with an analytical reference implementation in MATLAB, then develop a PyTorch-based U-Net to learn the mapping from charge density ρ to potential ϕ . This serves as a foundation for more complex particle-field simulations, including future extensions to 3D and electromagnetic fields from moving charges.

1 Introduction

The computation of electromagnetic fields from charged particle distributions is fundamental to many physics simulations, from plasma physics to molecular dynamics. Traditional approaches solve Poisson’s equation numerically using methods like finite differences, finite elements, or spectral methods. We explore whether a neural network can learn this mapping directly, potentially offering advantages in speed or flexibility.

1.1 Motivation

The eventual goal is a system where:

1. Charged particles scatter their state onto a spatial hierarchy (quadtree in 2D, octree in 3D)
2. A neural network processes this representation to derive electromagnetic fields
3. Particles gather field values to compute forces and update trajectories

Before tackling the full problem, we start with the simplest nontrivial case: learning the 2D electrostatic potential from static point charges.

1.2 Physics Background

The electrostatic potential in 2D from point charges is given by:

$$\phi(\mathbf{r}) = - \sum_i q_i \ln |\mathbf{r} - \mathbf{r}_i| \quad (1)$$

where we have absorbed physical constants for simplicity. This potential satisfies Poisson’s equation:

$$\nabla^2 \phi = - \frac{\rho}{\epsilon_0} \quad (2)$$

The electric field is the negative gradient of the potential:

$$\mathbf{E} = -\nabla \phi \quad (3)$$

A key property of Poisson’s equation is **linearity**: the potential from multiple charges equals the sum of potentials from individual charges (superposition principle).

2 MATLAB Reference Implementation

Before training a neural network, we developed a MATLAB implementation to compute the analytical solution and visualize the physics.

2.1 Implementation Details

The MATLAB script `matlab/electrostatic/electrostatic_potential_2d.m` performs the following:

1. **Particle Generation**: N point charges are placed randomly in a 2D domain $[0, 1]^2$, with charges $q_i \in [-1, 1]$ drawn uniformly at random.
2. **Charge Density (Scatter)**: Each particle deposits its charge onto a computational grid using nearest-cell assignment:

$$\rho_{ij} = \sum_p \frac{q_p}{\Delta x \Delta y} \cdot \delta_{i,i_p} \delta_{j,j_p} \quad (4)$$

where (i_p, j_p) is the grid cell containing particle p .

3. **Potential Computation**: The analytical potential is computed at each grid point:

$$\phi_{ij} = - \sum_p q_p \ln (\max(|\mathbf{r}_{ij} - \mathbf{r}_p|, \epsilon)) \quad (5)$$

where $\epsilon \approx 2\Delta x$ regularizes the singularity at particle centers.

4. **Electric Field**: Computed via numerical gradient:

$$\mathbf{E} = -\nabla\phi \approx - \left(\frac{\partial\phi}{\partial x}, \frac{\partial\phi}{\partial y} \right) \quad (6)$$

2.2 Visualization

The implementation produces three visualization panels:

1. **Charge Density**: Color map of ρ with particle positions marked (red for positive, blue for negative charges).
2. **Electrostatic Potential**: Color map of ϕ with equipotential contours overlaid. Positive charges create potential wells (low values), negative charges create peaks.
3. **Electric Field**: Magnitude $|\mathbf{E}|$ on logarithmic scale with streamlines showing field direction. Field lines emanate from positive charges and terminate at negative charges.

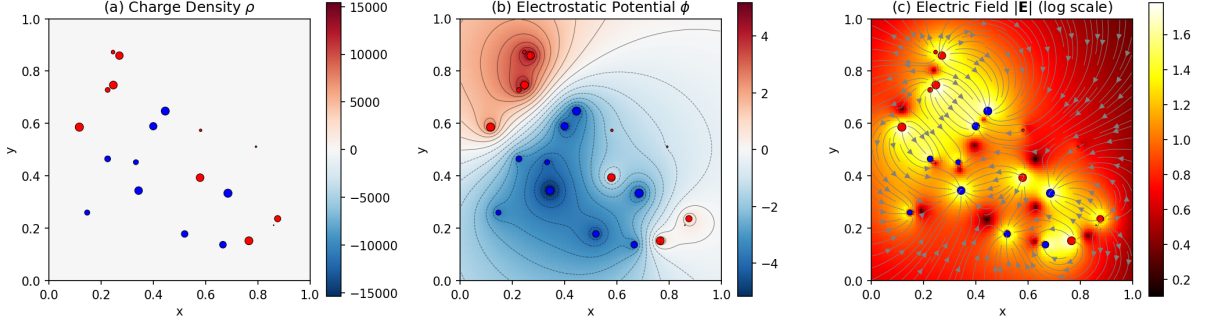


Figure 1: Visualization of the 2D electrostatic simulation with 20 random point charges. (a) Charge density ρ scattered to a 128×128 grid using nearest-cell assignment. (b) Electrostatic potential ϕ with equipotential contours. (c) Electric field magnitude $|\mathbf{E}|$ on logarithmic scale with streamlines showing field direction.

2.3 Verification

Two verification tests are performed:

2.3.1 Poisson's Equation

The numerical Laplacian $\nabla^2\phi$ is computed and compared to $-\rho$. For point charges, this comparison is approximate since the charge density involves delta functions, but away from particle centers the Laplacian should be near zero (as expected for a source-free region). Figure 2 shows this comparison.

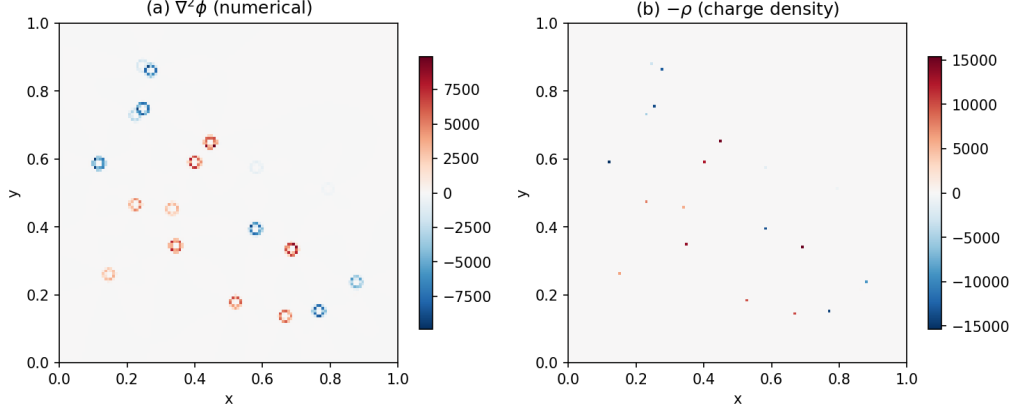


Figure 2: Verification of Poisson's equation. (a) Numerical Laplacian $\nabla^2\phi$ computed via finite differences. (b) Negative charge density $-\rho$. The Laplacian shows localized peaks at particle positions, matching the discrete charge distribution.

2.3.2 Superposition Principle

The particles are split into two groups A and B , and we verify:

$$\phi(A \cup B) = \phi(A) + \phi(B) \quad (7)$$

This test passes to machine precision (relative error $< 10^{-13}$), confirming the linearity of the implementation. Figure 3 demonstrates this by showing the potentials from each particle group separately and their sum.

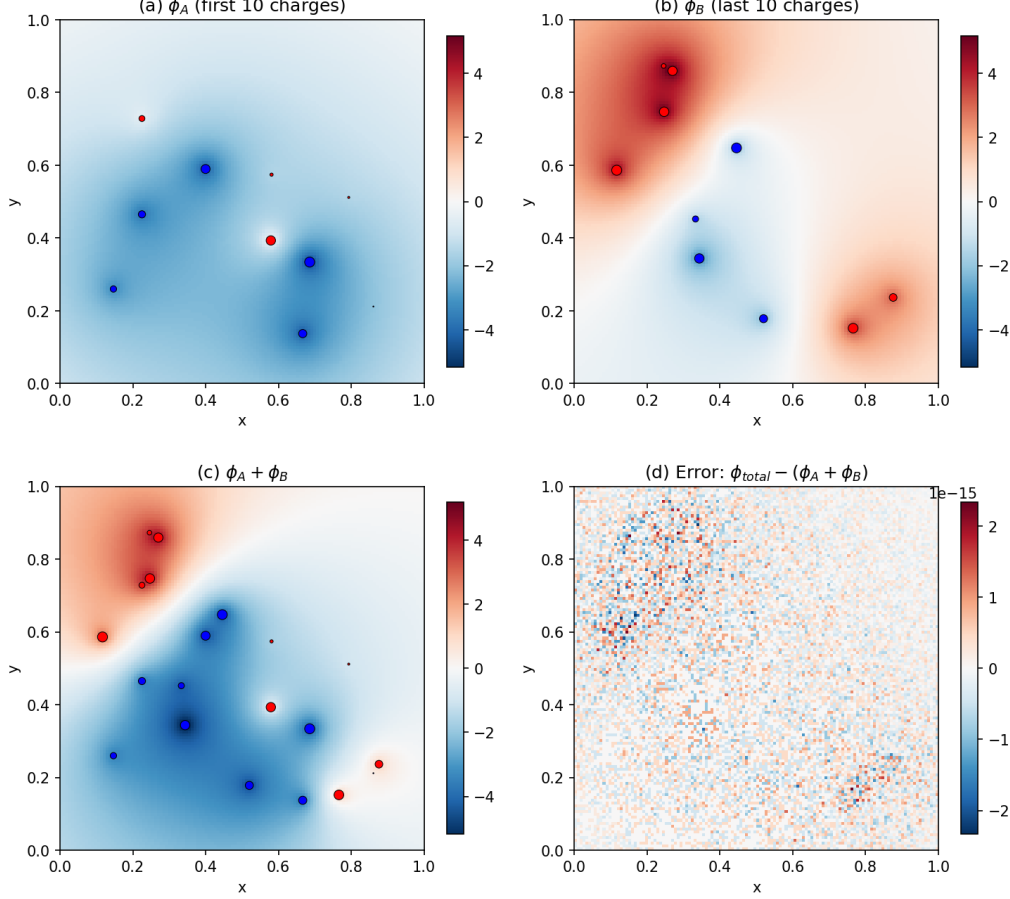


Figure 3: Verification of the superposition principle. (a) Potential from the first 10 charges (ϕ_A). (b) Potential from the last 10 charges (ϕ_B). (c) Sum of individual potentials ($\phi_A + \phi_B$). (d) Error between the direct computation and the sum, showing machine-precision agreement.

2.4 Parameters

Parameter	Value
Grid resolution	128×128
Domain	$[0, 1]^2$
Number of particles	20
Charge range	$[-1, 1]$
Regularization ϵ	$2/n_x$ (2 grid cells)

Table 1: Default parameters for simulation.

3 Python Implementation

The MATLAB reference implementation has been ported to Python for integration with PyTorch-based neural network training. The Python implementation is located in `electrostatic_unet/electrostatic` and provides the same functionality with a modular, object-oriented design.

3.1 Key Components

- **Particles:** Dataclass storing particle positions (x, y) and charges q

- `generate_particles()`: Creates random point charges in the domain
- `scatter_charges()`: Deposits particles onto grid using nearest-cell assignment
- `compute_potential()`: Computes analytical potential via superposition
- `compute_electric_field()`: Derives $\mathbf{E} = -\nabla\phi$ using numerical gradients
- `run_simulation()`: Orchestrates the full simulation pipeline

3.2 Code Example

```
from electrostatic_unet.electrostatic_2d import run_simulation,
    plot_fields

# Run simulation with 20 random charges
result = run_simulation(nx=128, ny=128, n_particles=20, seed=42)

# Access computed fields
phi = result['phi']          # Potential field (128, 128)
Ex, Ey = result['Ex'], result['Ey'] # Electric field components
rho = result['rho']          # Charge density

# Visualize results
plot_fields(result, save_path='output.png')
```

The Python implementation produces identical results to the MATLAB version, as verified by the superposition test passing to machine precision.

4 U-Net Architecture

The U-Net learns the mapping from charge density ρ to potential ϕ :

$$\phi = \text{UNet}(\rho; \theta) \quad (8)$$

4.1 Architecture Design

The U-Net follows the standard encoder-decoder structure with skip connections:

- **Input:** Single-channel image representing charge density $\rho(x, y)$
- **Encoder:** Sequence of convolution $\rightarrow$ activation $\rightarrow$ downsampling blocks, doubling channels at each level
- **Bottleneck:** Convolutional processing at coarsest resolution
- **Decoder:** Sequence of upsampling $\rightarrow$ skip connection concatenation $\rightarrow$ convolution blocks, halving channels at each level
- **Output:** Single-channel image representing potential $\phi(x, y)$

For a 128×128 input with 4 encoder levels, the spatial dimensions progress as:

$$128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \text{ (bottleneck)} \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$$

4.2 Why U-Net?

The U-Net architecture is well-suited for this problem because:

1. **Multi-scale processing:** The encoder captures both local charge structure and global potential behavior.
2. **Skip connections:** Preserve fine-grained spatial information needed for accurate potential near charges.
3. **Proven effectiveness:** U-Nets excel at image-to-image translation tasks with spatial correspondence between input and output.

5 Training Data Generation

Training data is generated using the same analytical formula as the MATLAB implementation:

1. Sample $N \sim \text{Uniform}(5, 50)$ particles per sample
2. Place particles uniformly in the domain (with margin from boundaries)
3. Assign charges $q_i \sim \text{Uniform}(-1, 1)$
4. Scatter charges to grid to create ρ
5. Compute analytical ϕ with regularization

Parameter	Value
Training samples	10,000
Validation samples	1,000
Grid resolution	128×128
Particles per sample	5–50 (uniform)

Table 2: Training data generation parameters.

6 Training

(To be completed after PyTorch implementation)

- Loss function: MSE between predicted and ground truth potential
- Optimizer: Adam
- Learning rate: 10^{-4}
- Batch size: 16–32
- Epochs: Until validation loss plateaus

7 Results

(To be completed after training)

8 Evaluation

Planned evaluation metrics:

1. **L2 Error:** Mean squared error on held-out test configurations
2. **Superposition Test:** Verify $\phi(A \cup B) \approx \phi(A) + \phi(B)$ for learned model
3. **Generalization:** Test on particle counts outside training distribution
4. **Visual Comparison:** Predicted vs ground truth potential fields

9 Future Work

- Extend to 3D with volumetric U-Net
- Replace regular grid with adaptive octree representation
- Add vector outputs for electric field $\mathbf{E} = -\nabla\phi$
- Introduce moving charges and learn magnetic field $\mathbf{B}$
- Couple to particle dynamics for full simulation loop

10 Conclusions

(To be completed)